De los datos al gráfico

Ing. en Sist. de Información
4° Año - Segundo Cuatrimestre

Repaso de clase · TP Pandas + TP Matplotlib

por qué hacemos esto

¿Qué información podemos obtener a partir de los datos que tenemos?

Esa es la pregunta que atraviesa las dos clases. No se trata de memorizar comandos: cada línea de código responde una decisión sobre los datos.



- TP de Pandas: datos reales de condición habitacional por provincia
- TP de Matplotlib: profundizamos cómo construir y controlar los gráficos
- Un mismo proceso de fondo en los dos trabajos

Ya sabemos cargar los datos. Ahora, ¿qué hacemos con ellos?

- Archivo: condicion_habitacional.xlsx
- Hoja de trabajo: Cuadro 1
- No hace falta repasar qué es Excel, un DataFrame o read_excel — eso ya lo conocemos

carga inicial

```
data = pd.read_excel(
    'condicion_habitacional.xlsx',
    sheet_name='Cuadro 1'
)
```

*Tener una tabla cargada **todavía no significa** que tengamos la información que necesitamos.*

Primero observamos.
Después decidimos.

Antes de transformar los datos necesitamos mirar cómo están organizados: qué representa cada columna y cuáles son los indicadores que vamos a usar.

primero conozco la estructura de los datos;
después decido qué transformación necesito.

¿Podemos resumir los distintos indicadores de cada provincia en un solo valor?

- En el TP resolvemos esto creando una nueva columna: total
- Pero aparece una decisión importante antes de escribir código

¿qué valores vamos a sumar para obtener ese total?

construimos una nueva variable

data['total']

nueva columna derivada

```
data['total'] = data.iloc[1:27, 1:13].sum(axis=1)
```

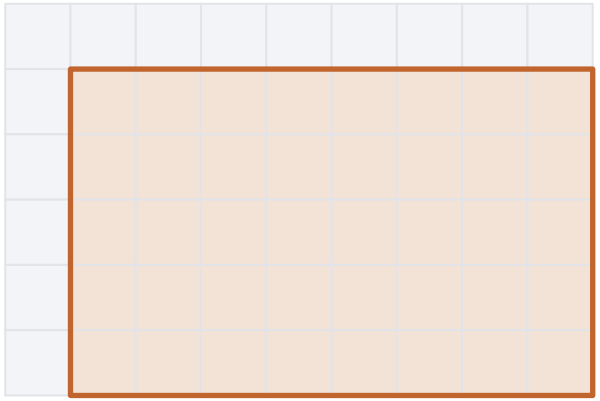
datos originales → transformación → **nuevo indicador**

Tomamos varios indicadores de cada provincia y construimos una nueva variable derivada de los datos originales. Esta idea aparece constantemente cuando trabajamos con datos.

dos preguntas antes de sumar

¿Qué parte de la tabla y en qué sentido?

`data.iloc[1:27, 1:13]`



filas 1→27 · columnas 1→13 (los indicadores)

Seleccionamos primero qué región de la tabla queremos usar: el primer rango son filas, el segundo, columnas. Lo importante no es el número — es decidir qué parte de la tabla corresponde a la operación.

`.sum(axis=1)`

```
ind. 1 + ind. 2 + ind. 3 + ... → total prov. 1
ind. 1 + ind. 2 + ind. 3 + ... → total prov. 2
ind. 1 + ind. 2 + ind. 3 + ... → total prov. 3
ind. 1 + ind. 2 + ind. 3 + ... → total prov. 4
```

axis=1 suma a lo largo de cada fila: para cada provincia, indicador + indicador + ... = total. Con otra orientación, la operación tendría otro significado.

el resultado

Ahora cada jurisdicción tiene un valor adicional

Jurisdicción	P. revest.	Cel. c/internet	...	total
Buenos Aires	...	...	...	...
Córdoba	...	...	...	...
Santa Fe	...	...	...	...
...	...	...	...	...

Ese valor no estaba originalmente como columna independiente.

Lo construimos nosotros a partir de los indicadores existentes.

En análisis de datos, muchas veces las variables que necesitamos no vienen dadas directamente. Tenemos que construirlas.

No eliminamos datos porque sí

```
quitar filas que no son provincias
data.drop(data.index[27:44], inplace=True)
data.drop(data.index[0], inplace=True)
```

Preparamos la tabla para que las operaciones posteriores trabajen sobre las filas que realmente representan las observaciones que queremos comparar.

Si graficamos antes de limpiar, el gráfico no sabe que esas filas no deberían estar — las incluye igual.

la secuencia completa

1. Cargar
2. Observar
3. Seleccionar
4. Limpiar
5. Transformar
6. Analizar

ordenar para comparar

¿Qué provincia tiene el mayor total?

de mayor a menor

```
data_ordenado = data.sort_values(
    'total',
    ascending=False
)
```

Ordenar nos permite pasar de una tabla donde las provincias aparecen en un orden arbitrario a una tabla organizada según la variable que queremos analizar.

nos quedamos con lo necesario

No siempre necesitamos mostrar todos los datos disponibles

```
data_ordenado_simple = data_ordenado[
    ['Jurisdicción', 'total']
]
```

provincia + **total**

Seleccionar las variables adecuadas también es parte del análisis: hace que la lectura sea más clara para la pregunta que estamos haciendo.

¿Qué tipo de gráfico nos ayuda a comparar provincias?

Con muchas provincias, comparar fila por fila en la tabla ya no es cómodo. Transformamos la tabla en una representación gráfica.

- Cada provincia: una categoría
- Cada barra: su valor de total
- La altura permite comparar rápido — ideal para comparar categorías

gráfico de barras

```
data_ordenado_simple.plot(
    x='Jurisdicción',
    y='total',
    kind='bar',
    title='Puntaje Total por Provincia',
    ylabel='Total',
    figsize=(9,6)
)
```

Los organiza para que podamos observarlos mejor

- Cuáles provincias presentan valores mayores
- Cuáles presentan valores menores
- Qué tan grandes son las diferencias
- Cómo se distribuyen los valores entre provincias

El gráfico no reemplaza los datos.
Los organiza para que podamos observarlos mejor.

guardar

```
plt.savefig('puntaje por provincia.jpg')
```

Guardar no es un detalle menor: el gráfico puede tener que incorporarse a un informe, una presentación o enviarse — no siempre queda solo en el notebook.

Barras versus líneas

BARRAS

- Categorías independientes
- Natural para comparar provincia A vs provincia B
- No sugiere ninguna secuencia entre ellas

LÍNEAS

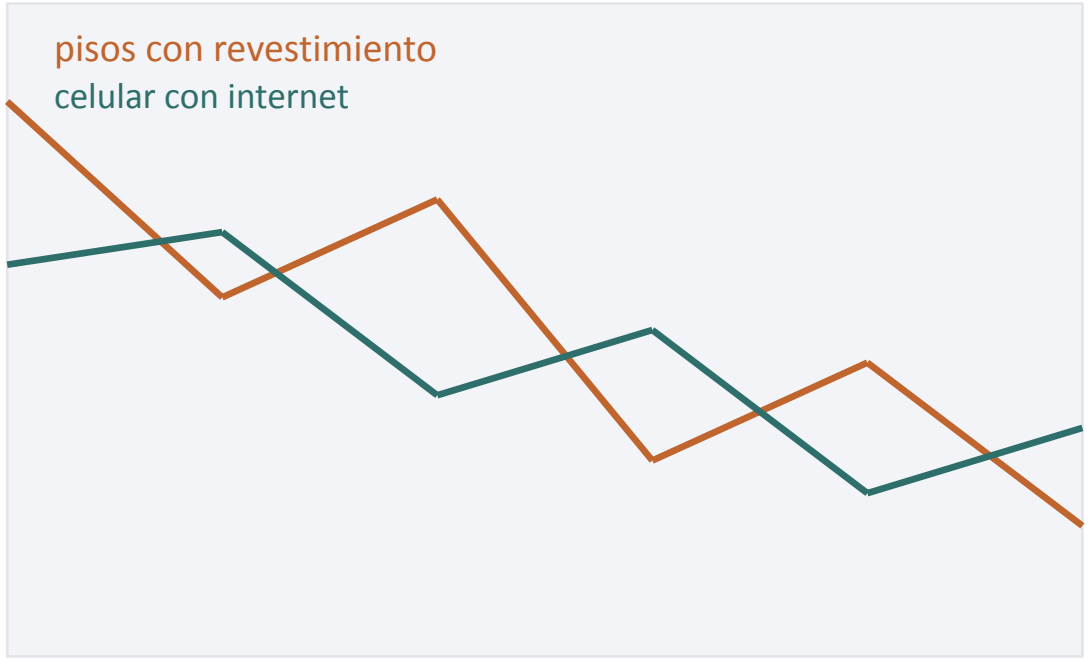
- Sucesión de puntos conectados
- Sugiere recorrido → variación → comportamiento
- Cuidado: las provincias no son una secuencia temporal — puede insinuar una evolución que no existe

El tipo de gráfico debe responder a la naturaleza de los datos y a la pregunta que queremos hacer.

Pisos con revestimiento vs. celular con internet

Dejamos de usar total. Ahora comparamos dos indicadores concretos para las mismas provincias — ya no calculamos un nuevo indicador, buscamos una relación visual.

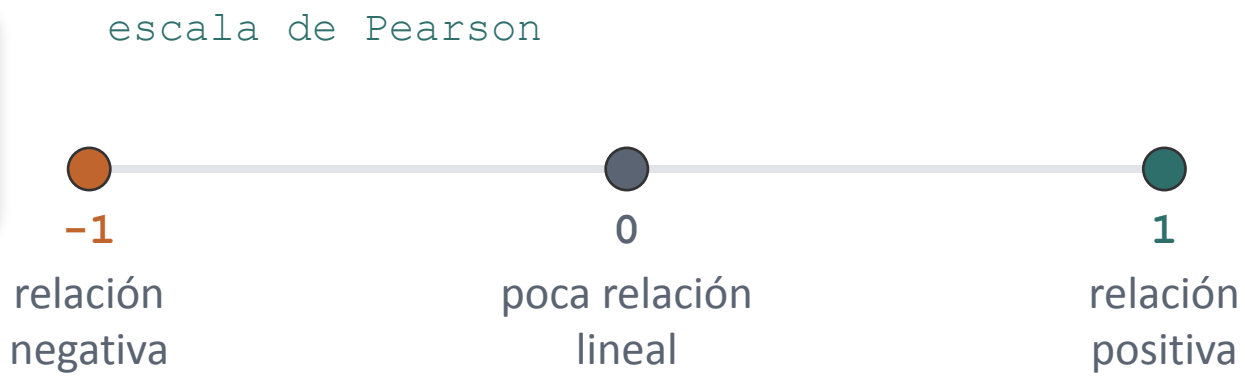
¿Se mueven parecido en algunas provincias, o presentan comportamientos muy diferentes?



¿Existe una relación entre los indicadores?

```
corr_data = data.corr(method='pearson')
```

Ya no miramos dos líneas: calculamos un valor que cuantifica la relación lineal entre variables numéricas.



La correlación da una medida numérica que complementa lo observado gráficamente.

Leer números en una matriz no resulta cómodo

data.corr(method='pearson') devuelve una matriz: cada fila y columna es una variable, y cada intersección, el coeficiente entre ambas. Con muchas variables, buscamos otra forma de representarlo.

```
annot=True → valor en cada celda

sns.heatmap(
    corr_data,
    annot=True,
    fmt='.2f'
)
```

1.00	0.80	0.30	-0.20	0.10
0.80	1.00	0.50	-0.10	0.20
0.30	0.50	1.00	0.40	-0.30
-0.20	-0.10	0.40	1.00	-0.60
0.10	0.20	-0.30	-0.60	1.00

Una correlación alta muestra una relación estadística.

No demuestra que uno provoque al otro.

Podemos encontrar relaciones entre distintos indicadores de condición habitacional. Eso nos permite formular preguntas y buscar patrones — pero para afirmar causalidad necesitaríamos otro tipo de análisis y evidencia.

¿Qué es la Correlación y Causalidad ?

La correlación es una relación estadística entre dos variables. Significa que cuando una variable cambia, la otra también tiende a cambiar (ya sea en la misma dirección o en la opuesta).

- **Correlación positiva:** Ambas variables aumentan o disminuyen juntas. *(Ejemplo: A más horas de estudio, mejores calificaciones).*
- **Correlación negativa:** Cuando una variable aumenta, la otra disminuye. *(Ejemplo: A mayor velocidad del auto, menor tiempo de viaje).*

La causalidad va un paso más allá. Significa que una variable produce un cambio directo sobre la otra. Existe una relación de causa y efecto.

Para que exista causalidad, se deben cumplir tres condiciones:

1. **Correlación:** Las dos variables deben estar asociadas.
2. **Prioridad temporal:** La causa debe ocurrir *antes* que el efecto.
3. **No espuria:** Se deben descartar otras explicaciones o variables ocultas.

otra tarea: unir información

Dos tablas compatibles no siempre están preparadas del mismo modo

- Hoja Cuadro 1.6
- Hoja Cuadro 1.7

```
df = pd.concat(
    pd.read_excel(
        'condicion_habitacional_concat.xlsx',
        sheet_name=['Cuadro 1.6','Cuadro 1.7']
    ),
    ignore_index=True
)
```

Dos tablas pueden tener información compatible pero no estar preparadas exactamente de la misma manera.

No alcanza con concatenar: primero hay que adaptar

Las dos hojas identifican a las provincias y sus capitales de forma distinta. Antes de unirlas necesitamos dejar esa información consistente.

identificamos el origen de los datos

```
data_cordoba.insert(0, 'Provincia', 'Cordoba')
```

Agregamos una columna que identifica el origen de esos datos — el mismo criterio que después aplicamos a Corrientes.

Cuadro 1.6
Córdoba

Cuadro 1.7
Corrientes

Corregir valores puntuales

corrección

```
# Cordoba: valor deliberadamente incorrecto
data_cordoba.iloc[[2],[1]] = 'Cordoba
Capitaaaal'

data_cordoba.replace(
    'Cordoba Capitaaaal', 'Cordoba Capital',
    inplace=True
)
```

corrientes

```
data_corrientes.insert(
    0, 'Provincia', 'Corrientes'
)

data_corrientes.replace(
    'Capital', 'Corrientes Capital',
    inplace=True
)
```

El ejemplo usa a propósito un valor incorrecto para mostrar la operación: localizar un dato, modificarlo, y dejar una denominación consistente entre las dos tablas.

```
# el resultado de todo el proceso
```

Preparar → hacer compatibles → unir → obtener una tabla conjunta

```
data_concatenado = pd.concat(
    [data_cordoba, data_corrientes],
    sort=False
)
```

1 Preparar tablas

2 Compatibilizar

3 Unir (concat)

4 Tabla conjunta

¿Cómo graficamos cuando nosotros mismos generamos los valores?

```
x = np.linspace(-10, 10, 50)
```

50 valores distribuidos uniformemente entre -10 y 10. Una función matemática no entrega sola una lista de puntos: generamos x, después calculamos y.

linspace vs range

- np.linspace(-10,10,50) → inicio, final, cantidad de valores
- list(range(-10,10)) → secuencia basada en pasos enteros
- Para puntos de una función que vamos a graficar, linspace resulta más conveniente

de una función a un gráfico

$y = a \cdot x + b$

```
a = -2
b = 3
y = a*x + b

plt.plot(x, y)
```

generamos X → aplicamos la función → obtenemos Y → graficamos X contra Y

un gráfico necesita contexto

`plt.title(...)`	*¿qué estoy viendo?*
`plt.xlabel(...)`	*¿qué representa X?*
`plt.ylabel(...)`	*¿qué representa Y?*
`plt.grid()`	*¿cómo ubico los valores?*

Graficar no es solamente dibujar una línea. Es comunicar información.

```
# cambiamos la función
```

$$y = a \cdot x^2 + b \cdot x + c$$

```
a = 1
b = -2
c = 3
y = a*x**2 + b*x + c
```

La relación entre x e y ya no es lineal. El gráfico cambia porque cambió la función que estamos representando.

`expresión matemática → valores → representación gráfica`

La tangente crece de forma muy distinta a seno y coseno

```
tita = np.linspace(-np.pi/2, np.pi/2, 500)
y = np.tan(tita)
```

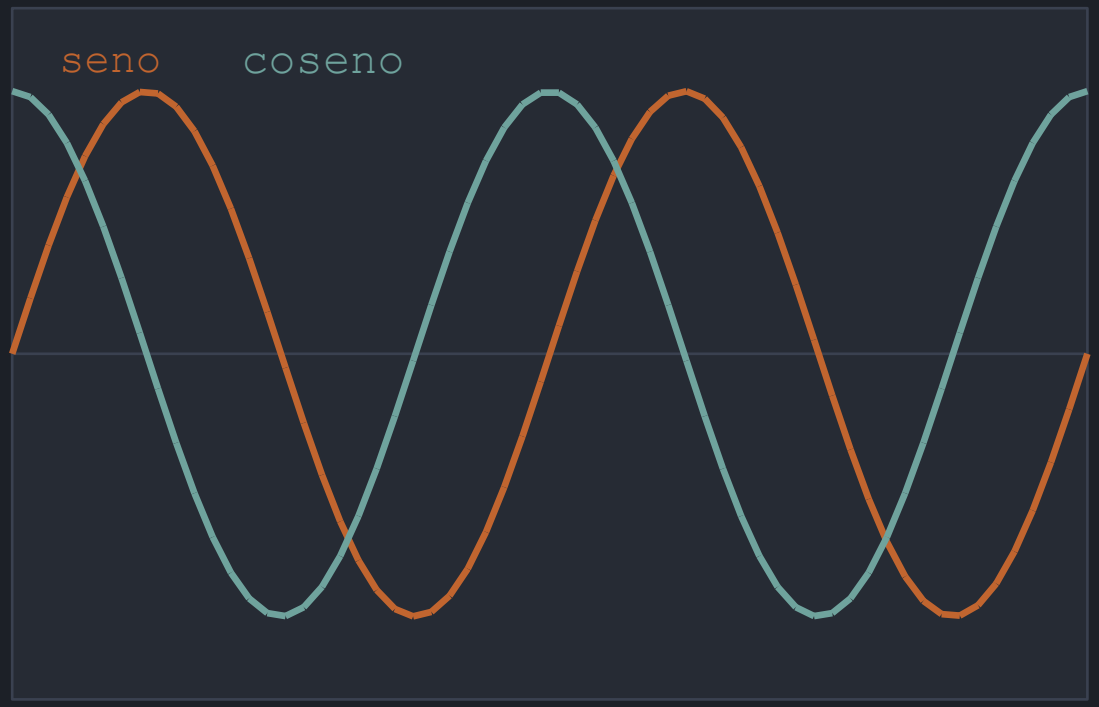
Cerca de determinados valores, sus resultados crecen enormemente. Por eso usamos límites en los ejes para observar la función de manera controlada.

los límites de los ejes también forman parte de la visualización

comparar dos series sobre el mismo eje

Seno y coseno juntos

```
x = np.linspace(0, np.pi*4, 100)
seno = np.sin(x)
coseno = np.cos(x)
```



Es muy parecido a lo que hicimos en Pandas al comparar pisos con revestimiento y celular con internet. La diferencia: acá nosotros generamos los datos.

personalizar no es decorar

Color · marcador · tipo de línea

```
plt.plot(x, seno, 'ro:')
plt.plot(x, coseno, 'g-')
```

- Título, etiquetas y leyenda
- Grilla y tamaño de la figura
- Texto y límites de los ejes

La personalización tiene sentido cuando ayuda a distinguir y comunicar las series — no cuando decora sin motivo.

subplot: comparar sin abrir cuatro figuras distintas

- Permite mostrar distintas funciones o vistas de un mismo problema
- Especialmente útil para comparar representaciones lado a lado

¿qué gráficos vale la pena colocar juntos y cuáles conviene separar?



el gráfico depende de la pregunta

Barras · histograma · scatter · pie

`plt.bar(...)`

Barras

Categorías con un valor cada una — simples, apiladas o agrupadas.

`plt.hist(...)`

Histograma

Cómo se distribuyen los valores numéricos agrupados en intervalos.

`plt.scatter(...)`

Scatter

Pares de valores — útil para detectar relaciones entre dos variables.

`plt.pie(...)`

Pie

Cómo se distribuye un total entre categorías.

Matplotlib no solo grafica rápido: también permite controlar todo

- Límites de X
- Límites de Y
- Etiquetas de los ticks
- Posición de los ejes
- Leyenda
- Líneas de referencia
- Puntos destacados

No alcanza con generar un gráfico rápido: cada uno de estos controles decide qué termina viendo quien lo lee.

volvamos a los datos de vivienda

Todo lo que hicimos con condición habitacional

01
Seleccionar

02
Limpiar

03
Calcular total

04
Ordenar

05
Elegir variables

06
Graficar

07
Comparar

08
Correlacionar

09
Combinar

dos formas de llegar al gráfico

Datos que ya teníamos vs. valores que generamos

TP PANDAS

datos reales → Pandas → **gráfico**

Trabajamos con información que ya existía y la fuimos preparando paso a paso.

TP MATPLOTLIB

función / valores → NumPy → **gráfico**

Nosotros generamos los valores desde una expresión matemática.

Ambos caminos terminan en el mismo concepto: tenemos datos y queremos representarlos de una manera que nos permita observar algo.

Matriz de selección de gráficos

Tipo de variable / datos	¿Qué quiero representar?	Gráfico recomendado	¿Qué me permite ver?	Ejemplo relacionado con los TPs
Cualitativa / categórica	Comparar categorías	**Barras**	Qué categoría tiene mayor o menor valor	Jurisdicción ⟶ total
	Comparar varias medidas dentro de cada categoría	**Barras agrupadas**	Diferencias entre indicadores para cada categoría	Provincia ⟶ Pisos con revestimiento vs. Celular con internet
	Mostrar cómo diferentes partes forman un total	**Barras apiladas**	Composición del total por categoría	Los ejemplos de barras apiladas del TP de Matplotlib
	Mostrar proporciones de un único total	**Pie / torta**	Qué porcentaje representa cada categoría	Ejemplo de plt.pie() del TP
Cuantitativa discreta	Comparar valores entre categorías o valores contables	**Barras**	Magnitudes individuales y diferencias	Una cantidad que queremos comparar entre categorías
	Analizar cómo se distribuyen los valores	**Histograma (si se agrupan en intervalos)**	Dónde se concentran los datos	Distribución de datos del ejercicio de plt.hist()
Cuantitativa continua	Analizar la distribución de los valores	**Histograma**	Concentración, dispersión y forma de la distribución	Datos numéricos generados en el TP
Dos variables cuantitativas	Analizar si existe relación entre ellas	**Scatter / dispersión**	Relación, tendencia y concentración de observaciones	plt.scatter(X, Y) del TP
Datos ordenados / evolución	Mostrar cambios y tendencias	**Líneas**	Cómo cambia una variable a medida que avanza el eje	Evolución del total por jurisdicción
Valores de una función respecto de otra variable	Mostrar el comportamiento de una función	**Líneas**	Forma y comportamiento de la función	sin(x), cos(x), tan(x), función lineal y cuadrática
Cualquier combinación anterior	Mostrar varios análisis simultáneamente	**Subplots**	Varias representaciones dentro de una misma figura	Ejercicio de 4 gráficos del TP

Cómo elegir un gráfico en 3 pasos

1 ¿Qué tipo de dato tengo?

- **Categórico**
- ↓
- **Discreto**
- ↓
- **Continuo**
- ↓
- **Dos variables numéricas**
- ↓
- **Datos ordenados / evolución**

2 ¿Qué quiero representar?

- **Comparar**
- **Distribución**
- **Relación**
- **Evolución**
- **Composición**
- **Proporción**
- **Comportamiento de una función**

3 ¿Qué representación responde mejor a esa pregunta?

Comparar categorías	→	**BARRAS**
Comparar varias medidas	→	**BARRAS AGRUPADAS**
Composición	→	**BARRAS APILADAS**
Proporciones	→	**PIE**
Distribución	→	**HISTOGRAMA**
Relación entre variables	→	**SCATTER**
Evolución / tendencia	→	**LÍNEAS**
Función matemática	→	**LÍNEAS**
Varios análisis	→	**SUBPLOTS**

la idea que nos llevamos

No alcanza con tener una tabla ni con generar un gráfico.

Tenemos que poder explicar: ***qué estamos calculando, por qué lo calculamos, qué estamos mostrando y qué significa lo que estamos viendo.***